

<div align="center">

EduML — Student Academic Performance Prediction System

A Gen Z-style interactive AI dashboard built with supervised ML pipeline simulation

Show Image

Show Image

Show Image

Show Image

Show Image

Show Image

 Zero dependencies · No build tools · Single HTML file · Works in any browser

</div>

Live Demo

 <https://YOUR-USERNAME.github.io/eduml-dashboard>

 Replace `YOUR-USERNAME` with your actual GitHub username.

Preview

Dashboard	Students	Analytics	Predict
KPIs + Charts	Card Grid + Drawer	4 ML Charts	Grade Prediction

Features

Dashboard Page

- Animated KPI counters (Total Students, Avg Performance, At-Risk, Model Accuracy)
- Performance trend line chart (6-month view)
- Grade distribution donut chart
- Live leaderboard — Top 5 students

- Real-time activity feed
- Quick stats grid

Students Page

- Live search by name or department
- Filter chips by Grade (A-D) and Department
- Glassmorphism student cards with stats
- **Sliding drawer** with full student profile on click
- One-click advisor notification

Analytics Page

- Feature importance horizontal bar chart
- 5-fold cross-validation accuracy (Train vs. Val)
- Weighted score histogram
- Attendance vs Performance scatter plot
- Model metrics: Accuracy 88.5% · F1 86.4% · ROC-AUC 0.923

Predict Page

- 8-feature input form
- Auto-computed engineered features (live update)
- One-click grade prediction (A / B / C / D / F)
- Animated confidence bar
- ML pipeline steps visualizer

ML Pipeline

```
python
```

Step 1 – Data Ingestion

```
df = pd.read_csv("students.csv") # 1,240 rows × 12 features
```

Step 2 – Missing Value Imputation

```
df.fillna(df.median(numeric_only=True), inplace=True)
```

Step 3 – Outlier Removal (IQR)

```
Q1, Q3 = df["score"].quantile([0.25, 0.75])
```

```
IQR = Q3 - Q1
```

```
df = df[(df["score"] >= Q1 - 1.5*IQR) & (df["score"] <= Q3 + 1.5*IQR)]
```

Step 4 – Feature Engineering

```
df["weighted_score"] = df["midterm"]*0.3 + df["assignments"]*0.25 + df["project"]
```

```
df["attendance_rate"] = df["attendance"] / 100
```

```
df["study_efficiency"] = (df["midterm"] + df["assignments"]) / (2 * df["study_hou
```

```
df["engagement_index"] = (df["attendance"] * 0.5 + df["participation"] * 0.5) / 1
```

Step 5 – Scaling

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Step 6 – Label Encoding

```
grade_map = {"A": 4, "B": 3, "C": 2, "D": 1, "F": 0}
```

```
df["grade_encoded"] = df["grade"].map(grade_map)
```

Step 7 – Train/Test Split

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, st
```

Step 8 – Model + Cross Validation

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.model_selection import cross_val_score
```

```
model = RandomForestClassifier(n_estimators=200, max_depth=8, min_samples_split=2,
```

```
cv_scores = cross_val_score(model, X_train, y_train, cv=5)
```

```
print(f"CV Accuracy: {cv_scores.mean():.2%} ± {cv_scores.std():.4f}")
```

```
# → CV Accuracy: 87.72% ± 0.0057
```

Step 9 – Hyperparameter Tuning (Grid Search)

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid = {
```

```
    "n_estimators": [50, 100, 150, 200, 300],
```

```
    "max_depth": [4, 6, 8, 10],
```

```
    "min_samples_split": [2, 3, 5]
```

```
}
```

```
grid = GridSearchCV(model, param_grid, cv=5, scoring="accuracy", n_jobs=-1)
```

```
grid.fit(X_train, y_train)
print(f"Best params: {grid.best_params_}")
print(f"Best accuracy: {grid.best_score_:.2%}")
# → Best params: {'max_depth': 8, 'min_samples_split': 2, 'n_estimators': 200}
# → Best accuracy: 88.52%
```

Model Results

Metric	Score
CV Accuracy (mean)	87.7%
Best Tuned Accuracy	88.5%
F1 Score	86.4%
ROC-AUC	0.923
CV Std Dev	±0.57

 Project Structure

```
eduml-dashboard/
|
├─ index.html      ← Entire dashboard (single-file app)
├─ README.md       ← This file
└─ LICENSE         ← MIT License
```

 Key Code Snippets

Grade Prediction Logic (JavaScript)

```
javascript
```

```

function runPredict() {
  const att = +document.getElementById('pa').value || 0;
  const mid = +document.getElementById('pm').value || 0;
  const asn = +document.getElementById('pasn').value || 0;
  const sh = Math.max(+document.getElementById('psh').value || 1, 1);
  const gpa = +document.getElementById('pgpa').value || 0;
  const prj = +document.getElementById('pprj').value || 0;

  // Weighted score (mimics the Python pipeline)
  const ws = mid * 0.3 + asn * 0.25 + prj * 0.25;

  // Bonus points (feature engineering)
  const bonus =
    (att > 85 ? 3 : att > 70 ? 1 : 0) +
    (sh > 20 ? 2 : sh > 12 ? 1 : 0) +
    (gpa > 8 ? 2 : gpa > 6 ? 1 : 0);

  const total = ws + bonus;

  // Grade threshold mapping
  const result =
    total >= 85 ? { g: 'A', l: 'Outstanding', cf: 94, r: 'Low' } :
    total >= 72 ? { g: 'B', l: 'Good', cf: 89, r: 'Low' } :
    total >= 58 ? { g: 'C', l: 'Average', cf: 82, r: 'Moderate' } :
    total >= 44 ? { g: 'D', l: 'Below Average', cf: 78, r: 'High' } :
    { g: 'F', l: 'Fail', cf: 85, r: 'Critical' };

  renderResult(result);
}

```

Feature Engineering (JavaScript)

javascript

```
function liveEF() {
  const a    = +document.getElementById('pa').value    || 0;
  const m    = +document.getElementById('pm').value    || 0;
  const asn  = +document.getElementById('pasn').value  || 0;
  const sh   = Math.max(+document.getElementById('psh').value || 1, 1);
  const gpa  = +document.getElementById('pgpa').value  || 0;
  const prj  = +document.getElementById('pprj').value  || 0;

  return {
    'Weighted Score':    (m*0.3 + asn*0.25 + prj*0.25 + (asn*0.5)*0.2).toFixed(1),
    'Attendance Rate':   (a / 100).toFixed(2),
    'Study Efficiency':  ((m + asn) / (2 * sh)).toFixed(2),
    'Engagement Index':  ((a * 0.5) / 100).toFixed(2),
    'GPA Level':         gpa >= 8 ? 'Strong' : gpa >= 6 ? 'Moderate' : 'Weak',
  };
}
```

CSS Design System

css

```
:root {
  --p:    #7c3aed;    /* Purple    - primary accent    */
  --c:    #06b6d4;    /* Cyan     - secondary accent    */
  --pk:   #f43f5e;    /* Pink     - danger / at-risk    */
  --g:    #10b981;    /* Green    - success / Grade A   */
  --am:   #f59e0b;    /* Amber    - warning / Grade C   */
  --t2:   #94a3b8;    /* Muted text
  --mu:   #4b5563;    /* Extra muted
  --bd:   rgba(255,255,255,.08); /* Border color
  --sf:   rgba(255,255,255,.04); /* Surface color
  --sfh:  rgba(255,255,255,.08); /* Surface hover
}
```

Chart.js Dark Theme Config

javascript

```
const DARK_DEFAULTS = {
  gridColor: 'rgba(255,255,255,.07)',
  tickColor: '#64748b',
  responsive: true,
  maintainAspectRatio: false,
  plugins: { legend: { display: false } },
};

// Example: Trend line chart
new Chart(ctx, {
  type: 'line',
  data: {
    labels: ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'],
    datasets: [{
      label: 'Avg Score',
      data: [71, 73, 75, 72, 78, 82],
      borderColor: '#7c3aed',
      backgroundColor: 'rgba(124,58,237,.12)',
      tension: 0.4,
      fill: true,
    }]
  },
  options: {
    ...DARK_DEFAULTS,
    scales: {
      x: { grid: { color: DARK_DEFAULTS.gridColor }, ticks: { color: DARK_DEFAULTS.tickColor } },
      y: { grid: { color: DARK_DEFAULTS.gridColor }, ticks: { color: DARK_DEFAULTS.tickColor } }
    }
  }
});
```

Local Setup

bash

1. Clone the repo

```
git clone https://github.com/YOUR-USERNAME/eduml-dashboard.git
```

2. Enter directory

```
cd eduml-dashboard
```

3. Open in browser – no server needed!

```
open index.html          # macOS
```

```
start index.html         # Windows
```

```
xdg-open index.html      # Linux
```

Optional: serve with a local server

```
npx serve .              # Node.js → http://localhost:3000
```

```
python3 -m http.server 8080 # Python → http://localhost:8080
```

Deploy to GitHub Pages

bash

Step 1 – Init and push

```
git init
```

```
git add .
```

```
git commit -m "🎓 EduML: Student Performance Dashboard"
```

```
git branch -M main
```

```
git remote add origin https://github.com/YOUR-USERNAME/eduml-dashboard.git
```

```
git push -u origin main
```

Step 2 – Enable GitHub Pages

GitHub Repo → Settings → Pages

Source → "Deploy from a branch"

Branch → main / / (root)

→ Click Save

→ Wait ~60 seconds

Step 3 – Your live URL

```
https://YOUR-USERNAME.github.io/eduml-dashboard
```

Customisation

Add a new student

javascript

```
// In index.html, find the S = [...] array and add:
{
  id: 10,
  n: 'Your Name',      // Full name
  d: 'CS',             // Department
  i: 'YN',             // Initials (2 chars)
  c: '#a855f7',        // Avatar color (any hex)
  att: 85,             // Attendance %
  mid: 80,             // Midterm score
  asn: 82,             // Assignment avg
  gpa: 8.5,            // Previous GPA (0-10)
  prj: 79,             // Project score
  g: 'B',              // Grade (A/B/C/D)
  r: 'Low',            // Risk (Low/Moderate/High)
  s: 82               // Overall score
}
```

Change color theme

css

```
/* In the <style> block, edit :root variables */
:root {
  --p: #your-primary-color;
  --c: #your-secondary-color;
  --pk: #your-danger-color;
  --g: #your-success-color;
}
```

Contributing

bash

```
# Fork → Clone → Branch → Commit → Push → PR
git checkout -b feature/my-feature
git commit -m "🚀 Add: my feature description"
git push origin feature/my-feature
# → Open Pull Request on GitHub
```

 License

MIT License

Copyright (c) 2026 YOUR-USERNAME

Permission is hereby granted, free of charge, to any person obtaining a copy of this software to deal in the Software without restriction, including the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software.

<div align="center">

Built with  using **Claude AI** by [Anthropic](#)

 **Star this repo if you found it useful!** 

Show Image

</div>